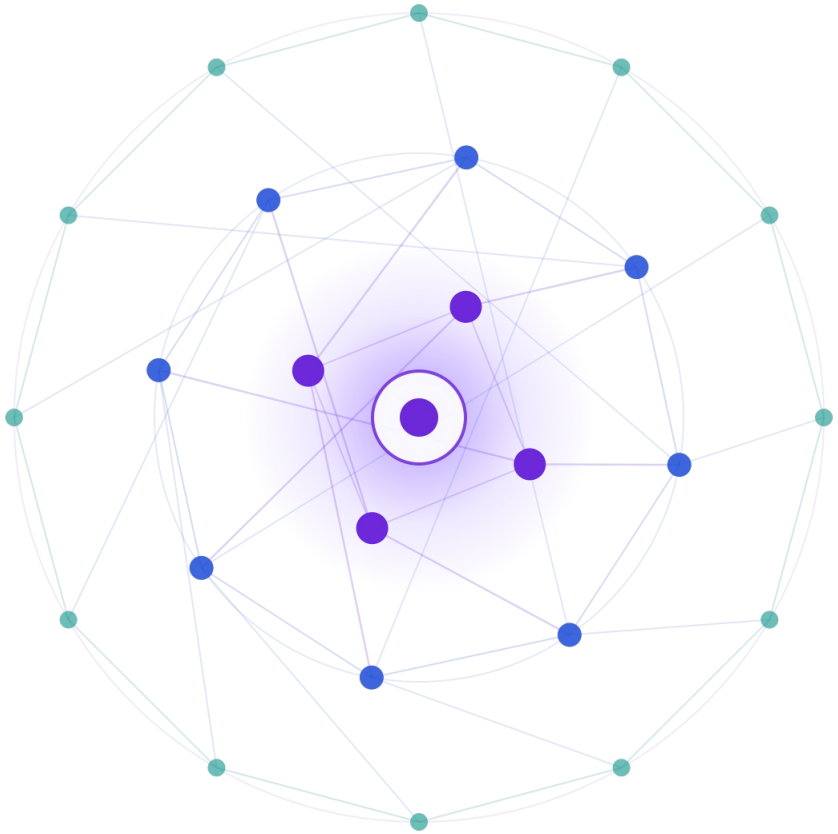
SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27



- Theme ID - 04 · Streaming Live RAG
- Project - Kairos: streaming live RAG that retrieves while you speak
- Team Name - VITV_Kairos_4
- College Name - VIT Vellore (VITV)
- Member Name & Email 1- Sarthak Trivedi, sarthak.trivedi2024@vitstudent.ac.in
- Member Name 2- Gurutheja Kavuturi
- Member Name 3- Nigam Kundu
- Submission Github link - github.com/CodeThread386/kairos-streaming-rag (tag PRISM_GENAI_HACKATHON_Y2026)

Theme

Theme 04, Streaming Live RAG: in a full-duplex voice conversation, retrieve the right context mid-conversation from one natural command.

It waits for silence

Batch RAG does nothing while the user speaks, then pays the whole retrieval round trip after they stop.

One query for many asks

"Venues for 30 in Pune, the cancellation policy and the catering options" is three questions. One embedding blurs them.

Late details restart it

"Actually, the trip was international." Restarting discards evidence, costs another full search and loses citations.

What we committed to

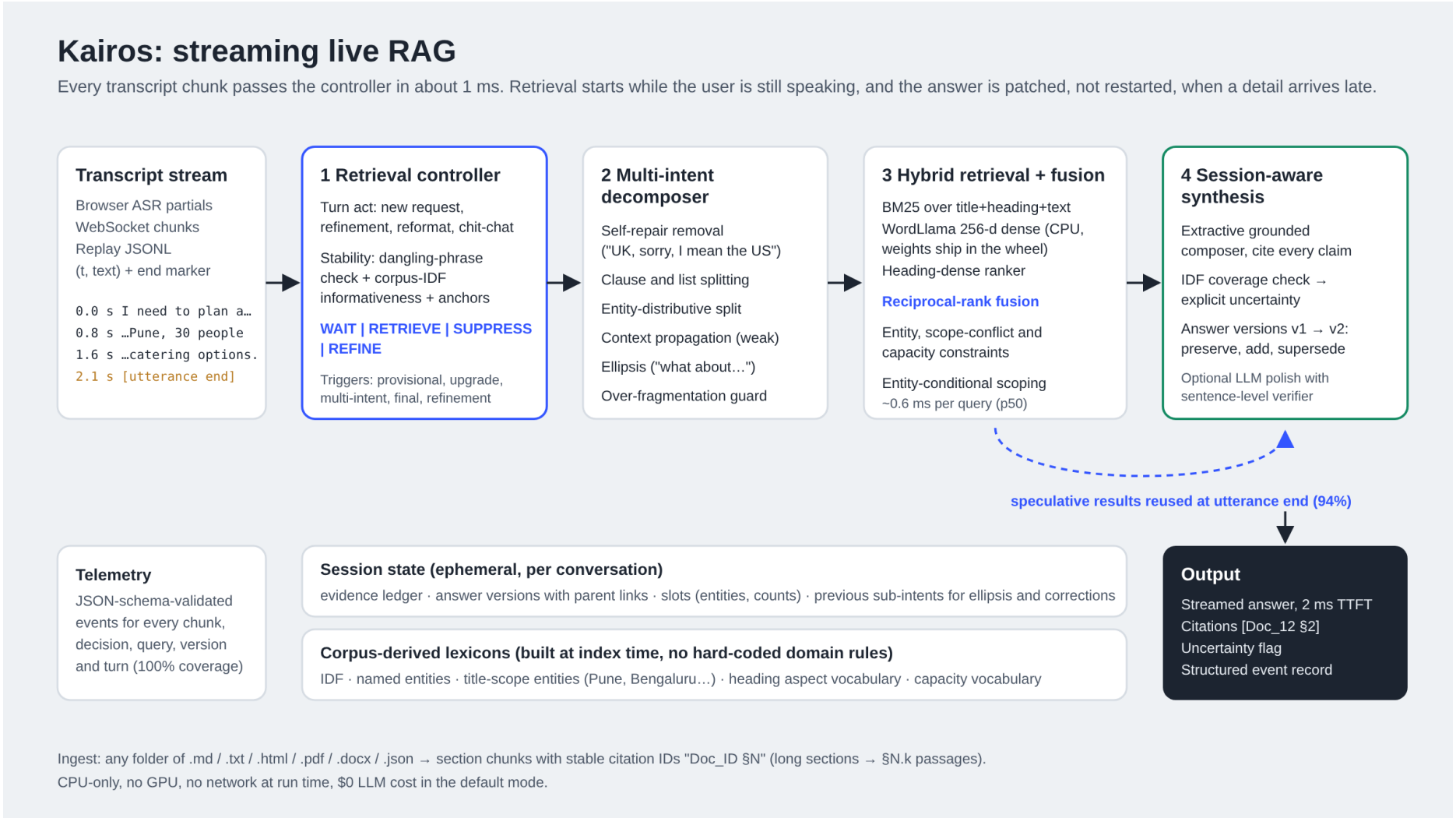
- **Listen incrementally:** decide on every chunk whether to wait, retrieve or skip
- **Decompose** one utterance into the questions it actually implies
- **Refine, never restart,** when a constraint arrives late
- **Ground every claim** in a cited section, or say it is unverified
- **Prove it:** measure all six acceptance gates G1–G6 on held-out streams

Existing Solutions & Gaps

Approach	How it works	Gap for live voice
Batch RAG (default chains in common RAG frameworks)	Waits for end of utterance, embeds it as one query, retrieves top-k	Retrieval starts only after the user stops; compound requests become one blurred query
LLM query rewriting / multi-query retrieval	An LLM rewrites or splits the question before searching	Adds an LLM round trip and cost per turn, still after the utterance ends
Realtime speech LLM APIs	Stream audio in and out with low latency	Tool or retrieval calls fire after the turn; grounding and citations are left to the app
Conversational RAG with history	Condenses chat history into a standalone query each turn	A late detail triggers a full new search; earlier evidence and citations are not carried forward

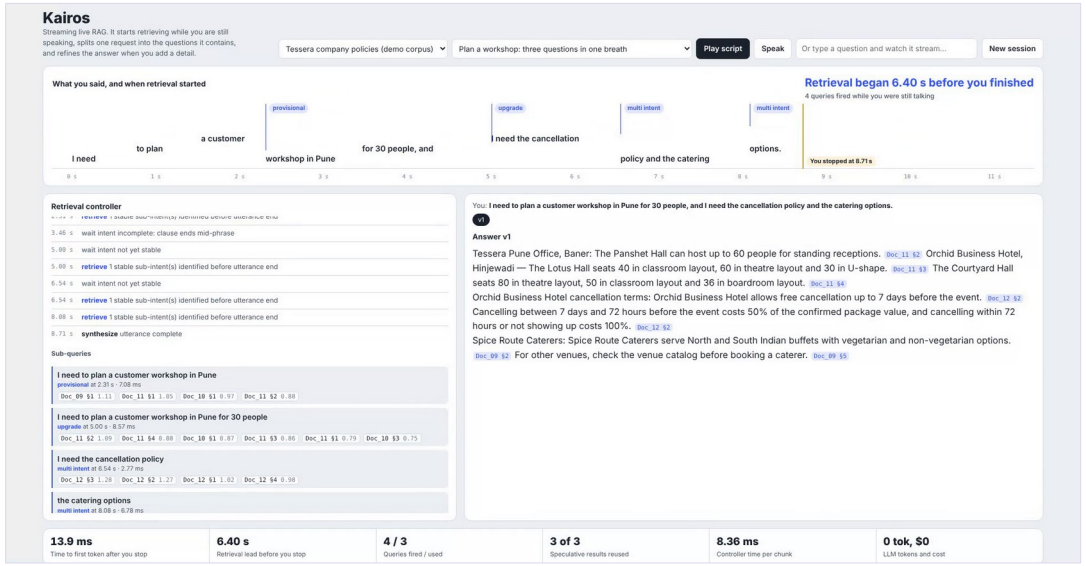
The gap: nothing decides when to retrieve while the user is still talking, splits the request cheaply, and patches a cited answer instead of restarting. Kairos does all three on a CPU, with no LLM in the loop.

Our Solutions & Architecture Diagram

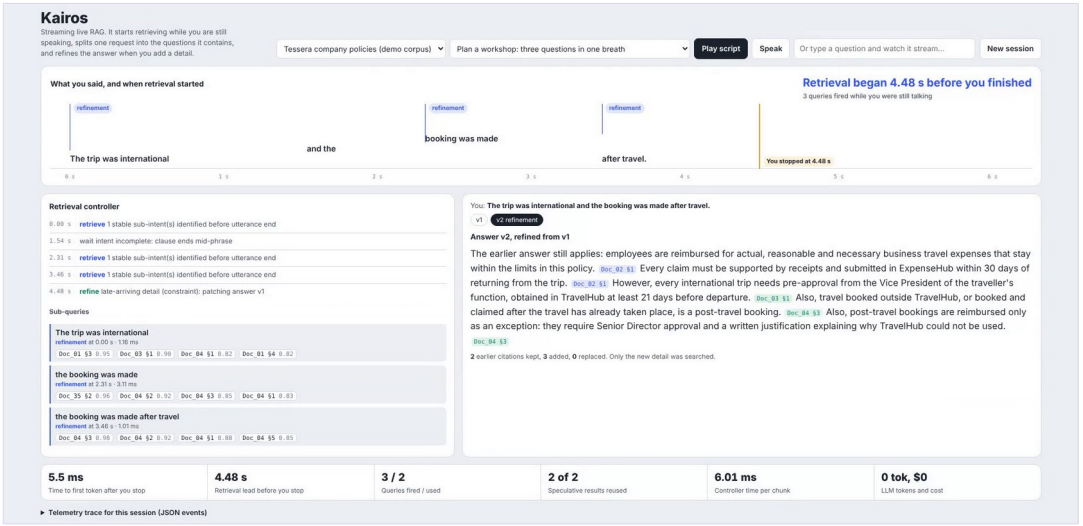


Controller ≈ 1 ms per chunk · retrieval ≈ 0.6 ms per query · answer starts ≈ 2 ms after the user stops · 0 LLM calls by default

Demo & Product Walkthrough



Three questions in one breath: retrievals fire 6.4 s before the speaker stops; answer starts 14 ms after.



Late detail "international, booked after travel": only the new details are searched; v2 keeps v1's citations.

Speech timeline

Words as spoken; blue markers are retrievals fired before the amber "you stopped" line

Controller log

Every WAIT / RETRIEVE / SUPPRESS decision with its reason and each sub-query's hits

Versioned, cited answer

Citation chips open the source sentence; v2 lists kept, added and replaced claims

Live telemetry

Time to first token, retrieval lead, queries fired and used, reuse, cost

Demo video (3:30): demo/kairos_demo.mp4 in the repository; YouTube link in the README

Tools and tech stack used

Streaming core

Python 3.12 · asyncio · FastAPI + WebSocket · Uvicorn

Retrieval

BM25 (own implementation) · WordLlama 256-d static embeddings (CPU, ships in the wheel) · reciprocal-rank fusion · NumPy

Language processing

Snowball stemmer · corpus-derived lexicons (IDF, entities, scopes, aspects) · rule-based self-repair and clause segmentation

Ingestion

Markdown / HTML / TXT / JSON parsers · pypdf · python-docx → section chunks with Doc_ID §N

Interface

Single-page console (HTML + JS) · Web Speech API for live mic input · typed input streamed word by word

Quality & ops

pytest (16 tests) · jsonschema telemetry validation · replay benchmark harness · Docker + compose · GitHub Actions CI

Optional LLM

Any OpenAI-compatible endpoint (Ollama, vLLM, OpenAI, Gemini, Groq) with a sentence-level grounding verifier

Demo production

Playwright (scripted UI recording) · Kokoro-82M TTS via sherpa-onnx (offline narration) · ffmpeg

Impact & Use case

Galaxy voice assistant

"Turn on Wi-Fi calling and how do I stop apps draining battery" retrieved from device manuals while the user is still speaking.

Enterprise help desks

HR, travel and IT policy questions with several asks per sentence, answered with citations auditors can check.

In-car and hands-free

Drivers add details mid-request ("actually, the route via the highway"). The answer is patched, not restarted.

Live agent assist

Surfaces cited policy snippets to a call-centre agent before the customer finishes the sentence.

2.4 ms

time to first token after the user stops, even when each search takes 150 ms (batch RAG: 152 ms)

+27 pts

answers that cite the right section, vs batch RAG (62.7% → 89.5%)

$0 / turn

no LLM in the loop; 16 MB embedding model, CPU only, so it fits on device

Innovation highlights, results and limitations

Innovation

- Retrieval controller with dangling-phrase and corpus-IDF stability checks: early retrieval with 0% false triggers
- Speculative retrieval reused only when it covers every final word (93–94% reuse)
- Weak context propagation and entity-conditional scoping across sub-queries
- Versioned answers: preserved, added and superseded claims, plus zero-query reformatting
- Extractive grounding by construction, plus a verifier that keeps an optional LLM honest

Limitations

- Synonym gaps ("cap" vs "limit") cause 4–6% false "unverified" flags
- Multi-hop joins (city → tier → rate) are cited, not computed
- Harder on real text: 72% evidence recall on the untuned 18F handbook

Gate (Theme 4)	Target	Test-1	Clean test-2
G1 one-command run, unattended replay	pass	pass	pass
G2 early retrieval	≥ 80%	100%	100%
G3 multi-intent identification	≥ 70%	100%	100%
G4 citation support / fabricated	≥ 85%	100% / 0	100% / 0
G5 refinement continuity	100%	100%	100%
G6 telemetry coverage	100%	100%	100%
Evidence recall (baseline 62.7%)	–	89.5%	95.8%
Key-fact recall (baseline 62.3%)	–	85.5%	83.3%

Test-2 was frozen before any error analysis and never used for tuning. Full report: reports/metrics.md.

What’s next

On-device for Galaxy

Port the controller and index to Android (ONNX / Kotlin); the 16 MB embedding model and ~1 ms decisions already fit a phone budget.

Audio-native endpointing

Use VAD and prosody, not only transcript punctuation, to time retrieval and handle barge-in during the spoken answer.

Close the synonym gap

A small cross-encoder, or the optional LLM, used only as a verifier when coverage is borderline (≈20–50 ms).

Learn the controller

Train the wait/retrieve policy on logged traces to trade lead time against wasted queries per user population.

Multilingual

Hindi, Korean and code-mixed speech with multilingual static embeddings and language-aware segmentation.

Salient entities on real corpora

Filter link text and boilerplate before entities can scope a query (the main 18F failure mode).

Brownie points slide(differentiation)

Reproduces the guide, line by line

All three worked examples in the Theme 4 guide replay as specified: provisional retrieval at 0.8 s, refinement without restart, reformatting with retrieval_required: false.

Honest measurement

Frozen splits with SHA-256 hashes, a clean test-2, a real untuned corpus, the first pre-analysis run kept, and every failure listed.

Nothing hard-coded

Entities, scopes, aspects and capacity words are mined from the corpus; the same code runs on the 18F handbook.

Latency hiding, quantified

With a 150 ms retrieval stack, Kairos starts answering in 2.4 ms against 152 ms for batch RAG (replayed traces).

Zero-cost by default, LLM-ready

CPU only, $0 per turn; an optional LLM is wrapped in a verifier that drops fabricated citations (tested).

Production hygiene

One-command Docker, CI building the image and running the replay suite, schema-validated traces, 16 tests.

Checklist- Updated on Public GitHub

- Working prototype code — public GitHub repo: Y (github.com/CodeThread386/kairos-streaming-rag)
- README with reproducible setup instructions: Y (one-command Docker run, local steps, API and CLI examples)
- Demo video, max 5 minutes: Y (demo/kairos_demo.mp4, 3 min 30 s, in the tagged commit)
- Presentation file (PPT or PDF): Y (presentation/VITV_Kairos_Submission.pptx + .pdf)
- AI disclosure: Y (docs/AI_DISCLOSURE.md + filled LangAI3.0 form)
- requirements.txt, Dockerfile, docker-compose.yml, tests, benchmark report: Y
- Release tag PRISM_GENAI_HACKATHON_Y2026 on the final commit: Y

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India